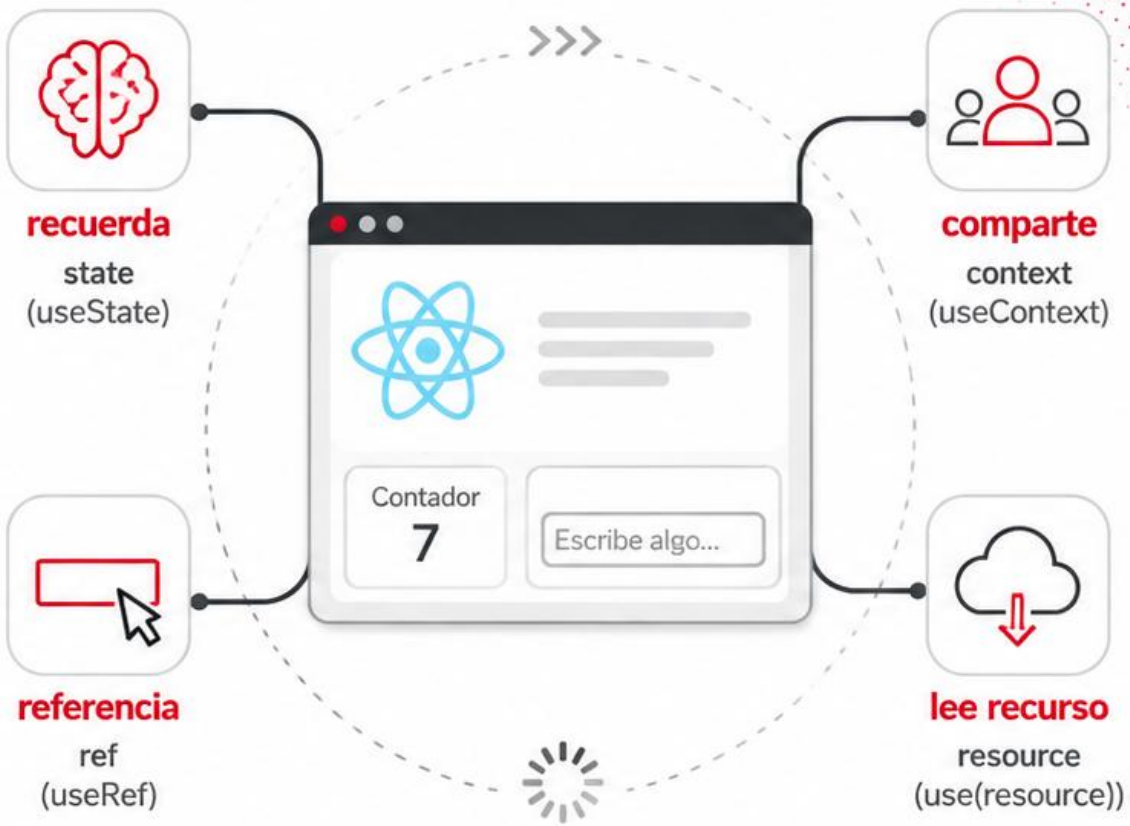
React con memoria, contexto, referencias y recursos

Semana 13 • Hooks y acceso a recursos

- **Curso:** JavaScript Avanzado.
- **Unidad 3:** React.
- **Semana** 13.
- **Docente:** Mg. Ivan Eduardo Anchante Reyes.
- **Ejes de la sesión:**

state Hooks · context Hooks · ref Hooks · acceso a recursos con use(resource).

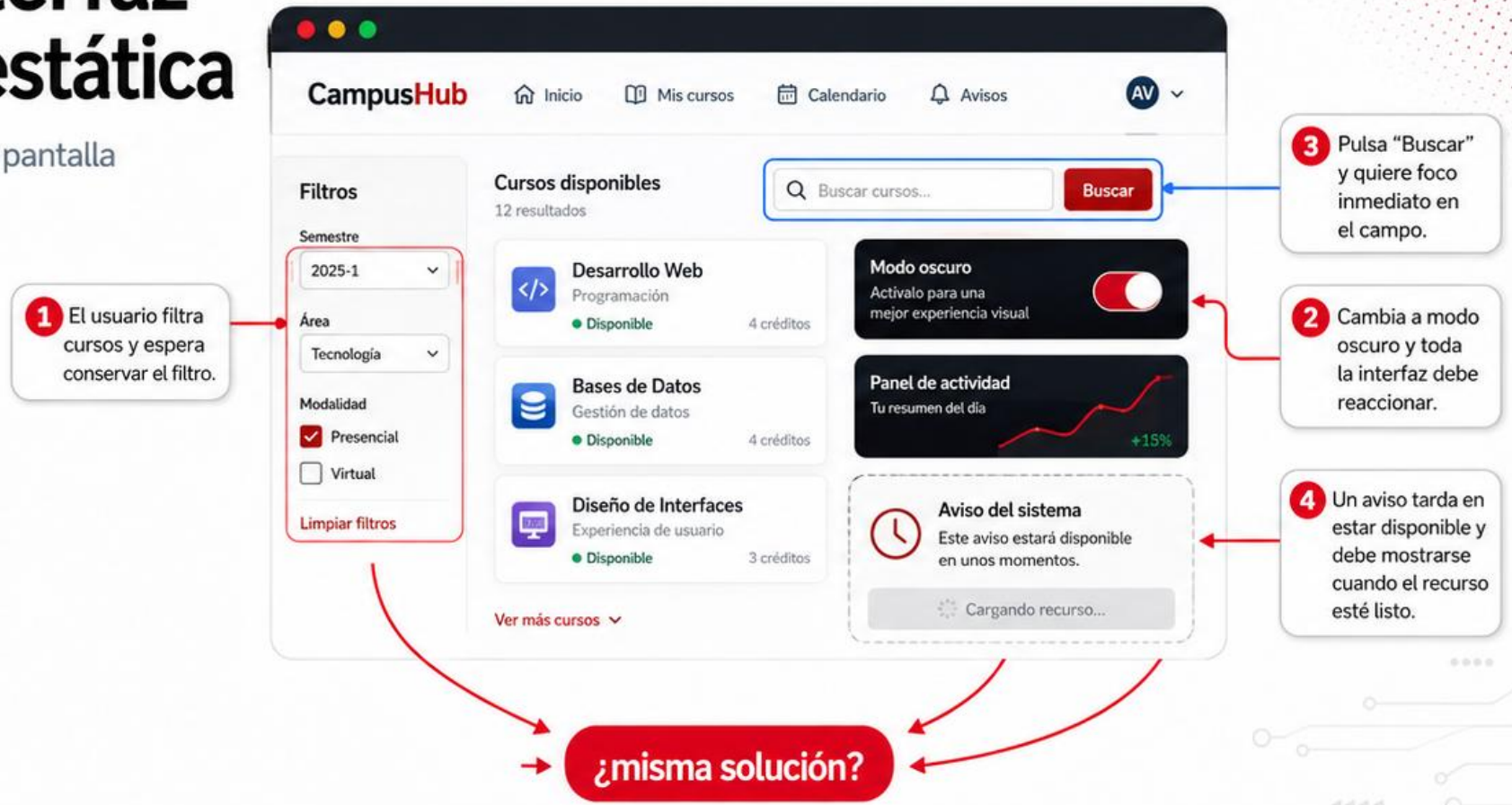


Desafío: la interfaz ya no puede ser estática

Cuatro necesidades aparecen en una sola pantalla

Escenario CampusHub: 1) el usuario filtra cursos y espera conservar el filtro; 2) cambia a modo oscuro y toda la interfaz debe reaccionar; 3) pulsa "Buscar" y quiere foco inmediato en el campo; 4) un aviso tarda en estar disponible y debe mostrarse cuando el recurso esté listo.

Pregunta: ¿resolverías las cuatro necesidades con la misma herramienta?



Universidad Tecnológica del Perú

Recuperamos la Semana 12

Componentes, navegación y formularios ya son la base

React • componentes • props • React Router • formularios con onSubmit + FormData.

Semana 12 → **Semana 13**



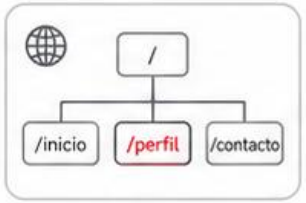
componentes

Construimos piezas reutilizables de UI.

```
<Perfil
  nombre="Ana"
  edad={20}
  activo={true} />
```

props

Pasamos datos de padre a hijo.



React Router

Navegamos entre vistas sin recargar.



formularios con onSubmit + FormData

Capturamos y enviamos datos del formulario.

interactividad persistente

La próxima semana daremos el siguiente paso.

Preguntas: ¿qué parte de la interfaz era reutilizable? • ¿qué información llegaba mediante props?
• ¿cómo capturamos el formulario sin Hooks?

Saberes previos: ¿qué debe recordar React?

Analiza cada situación. Elige la herramienta principal y explica por qué.

Opciones: useState · useContext · useRef · use(resource)

A



La categoría 'Frontend' seleccionada cambia la lista de cursos.

Mi elección: _____

B



El tema oscuro se utiliza en Header y Dashboard.

Mi elección: _____

C



Un botón debe colocar el cursor en el buscador.

Mi elección: _____

D



Un aviso se recibe mediante una Promise estable.

Mi elección: _____

Universidad Tecnológica del Perú

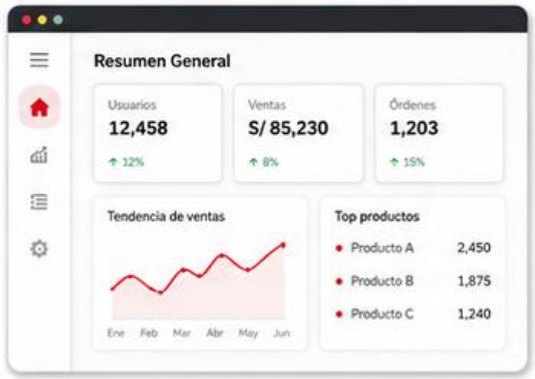
¿Para qué sirven los Hooks en una aplicación real?

Interacción, coordinación y control sin abandonar componentes funcionales

Pregunta al grupo: ¿qué problema profesional resuelve una interfaz que puede recordar filtros, compartir tema/usuario, enfocar controles y manejar contenido que todavía no está listo?

Aplicaciones: dashboards · formularios · e-commerce · soporte · administración · portales internos.

Dashboards



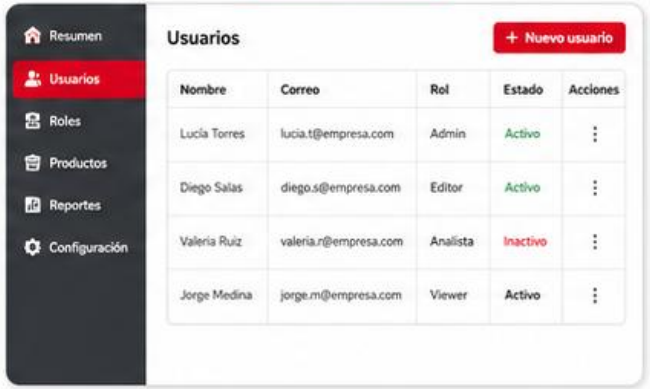
E-commerce



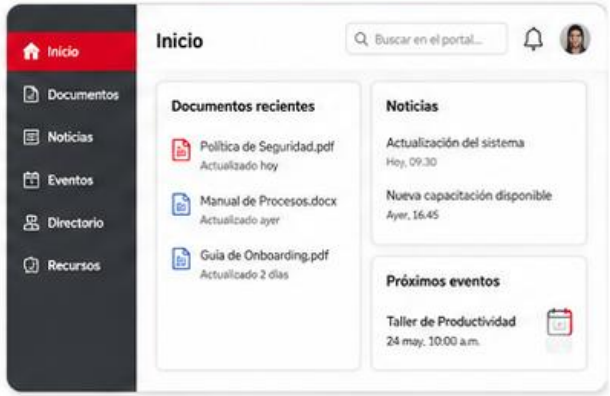
Formularios



Administración



Portales internos



Soporte



Universidad Tecnológica del Perú

Logro y ruta de la sesión

Elegir la herramienta adecuada y demostrarla funcionando

Al finalizar la sesión, el estudiante implementa una aplicación React interactiva utilizando **useState, useContext, useRef** y acceso a recursos con **use(resource)**, a partir de un caso funcional y **sin consumir servicios REST.**

sin REST esta semana

Ruta: estado → contexto → refs → recursos → integración → práctica individual.

1. **estado (useState)**
   Gestiona datos locales y dispara re-render.
2. **contexto (useContext)**
   Comparte datos entre componentes sin prop drilling.
3. **refs (useRef)**
   Accede a elementos del DOM o valores persistentes.
4. **recursos (use(resource))**
   Lee recursos de forma declarativa y reactiva (sin fetch manual).
5. **integración**
   Conecta estado, contexto, refs y recursos en un flujo coherente.
6. **práctica individual**
   Construye tu aplicación React funcional y completa.

Producto final: interfaz React funcional



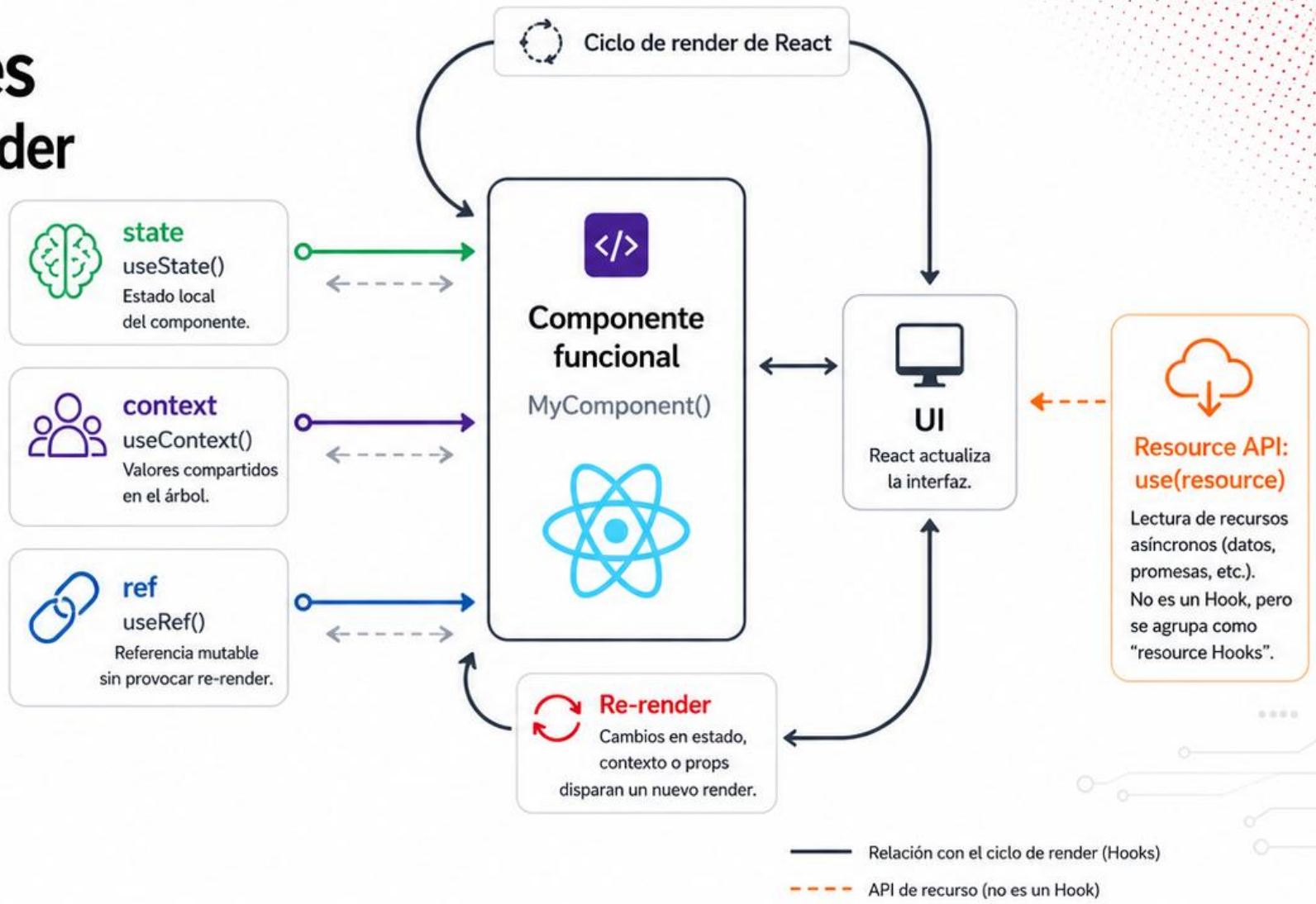
Hooks: capacidades conectadas al ciclo de render

No son funciones cualquiera

Hook = función de React cuyo nombre comienza con "use" y que permite usar capacidades de React desde componentes o custom Hooks.

Ejemplos de hoy: **useState** · **useContext** · **useRef**.

Nota: use(resource) es una API de recurso documentada por React y no es técnicamente un Hook, aunque el sílabo la agrupa dentro de "resource Hooks".



Reglas de Hooks: el orden importa

Solo en el nivel superior y desde funciones de React.

1. **Regla 1:** llama Hooks en el nivel superior del componente o custom Hook.
2. **Regla 2:** llama Hooks solo desde componentes de React o custom Hooks.
3. **Evita:** if · for · while · funciones anidadas · try/catch.
4. **Excepción conceptual:** use(resource) puede llamarse condicionalmente porque no es un Hook.

CORRECTO

Orden estable y llamadas en el nivel superior

```
function Panel() {
  const [filtro, setFiltro] = useState("Todos");
  const tema = useContext(ThemeContext);
  const inputRef = useRef(null);
  return <div />;
}
```

INCORRECTO

Orden inestable: Hook dentro de una condición

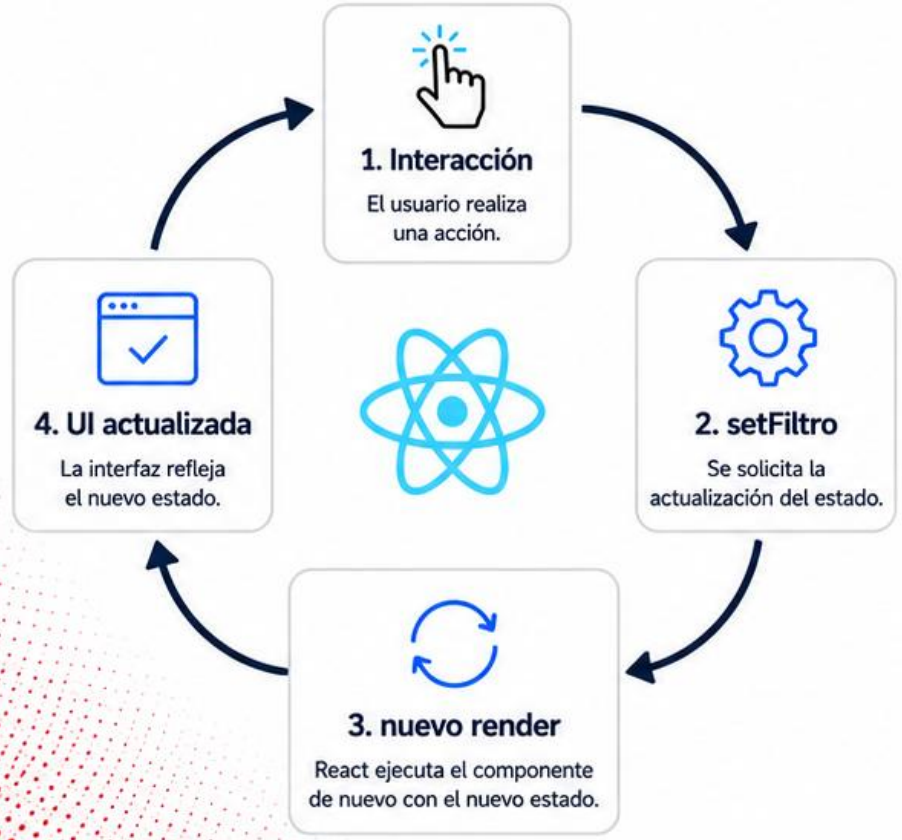
```
function Panel({ activo }) {
  if (activo) {
    const [x, setX] = useState(0);
  }
  return <div />;
}
```

Nota: use(resource) puede llamarse condicionalmente porque no es un Hook.

Universidad Tecnológica del Perú

useState: React recuerda y vuelve a renderizar

Estado = información que participa del resultado visual.




```
const [filtro, setFiltro] = useState("Todos");
```

`filtro` = valor actual

`setFiltro(nuevoValor)` = solicita actualización

cuando cambia el estado, **React** vuelve a renderizar el componente.

Regla: si el dato cambia lo **que ves**, probablemente es candidato a **state**.

Universidad Tecnológica del Perú

Demostración: un filtro que sí cambia la interfaz

Estado → evento → render derivado

- CampusHub: botones “Todos · Frontend · Backend” actualizan filtro mediante setFiltro.
- La lista visible se deriva del estado actual.
- Predicción antes de ejecutar: si el filtro cambia a “Frontend”, ¿qué componentes deberían desaparecer y cuáles permanecer?

CÓDIGO (evento + estado)

Interfaz: botones que actualizan el filtro

Todos | Frontend | Backend

Código (React)

```jsx
import { useState } from "react";
export default function Filtros() {
  const [filtro, setFiltro] = useState("Todos");
  return (
    <div className="space-x-2">
      <button onClick={() => setFiltro("Todos")} className="btn">Todos</button>
      <button onClick={() => setFiltro("Frontend")} className="btn btn-primary">Frontend</button>
      <button onClick={() => setFiltro("Backend")} className="btn">Backend</button>
    </div>
  );
}
```

Estado: `filtro ∈ {"Todos",""Frontend",'"Backend"}`

Evento: `onClick` → **setFiltro**(nuevoValor) → dispara render → lista derivada

RESULTADO (render derivado)

ANTES (filtro = “Todos”)

Cursos disponibles

- Desarrollo Web (Frontend) — React · Componentes · Hooks — Publicado
- JavaScript Avanzado (Frontend) — ES6+ · Async · DOM — Publicado
- CSS Moderno (Frontend) — Flexbox · Grid · Animaciones — Publicado
- Node.js (Backend) — Express · APIs · Middleware — Publicado
- Bases de Datos (Backend) — SQL · Modelado · Consultas — Publicado

DESPUÉS (filtro = “Frontend”)

Cursos disponibles

- Desarrollo Web (Frontend) — React · Componentes · Hooks — Publicado
- JavaScript Avanzado (Frontend) — ES6+ · Async · DOM — Publicado
- CSS Moderno (Frontend) — Flexbox · Grid · Animaciones — Publicado
- Node.js (Backend) — Express · APIs · Middleware — Publicado
- Bases de Datos (Backend) — SQL · Modelado · Consultas — Publicado

Solo permanecen los cursos Frontend.
Los cursos Backend desaparecen (render derivado del estado **filtro**).

Cuando el siguiente estado depende del anterior

Usa la forma funcional del setter




```
setFavorito(anterior => !anterior);
```

```
setCantidad(c => c + 1).
```

Si el nuevo valor depende del anterior, la función **updater** expresa esa dependencia de forma explícita.

Pregunta: ¿qué ventaja tiene frente a calcular con un valor capturado anteriormente?

Microejercicio: ¿qué debería ser estado?

Decide antes de programar

Caso: una galería de proyectos permite elegir una categoría, muestra el nombre del docente, se adapta al ancho de pantalla y tiene un buscador.

A **Categoría seleccionada**



¿?

B **Nombre del docente**



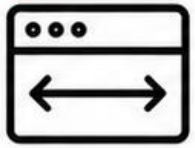
¿?

C **Nodo del buscador**



¿?

D **Ancho de ventana**



¿?

Pregunta: ¿qué dato guardarías como estado en este caso? Explica tu criterio.

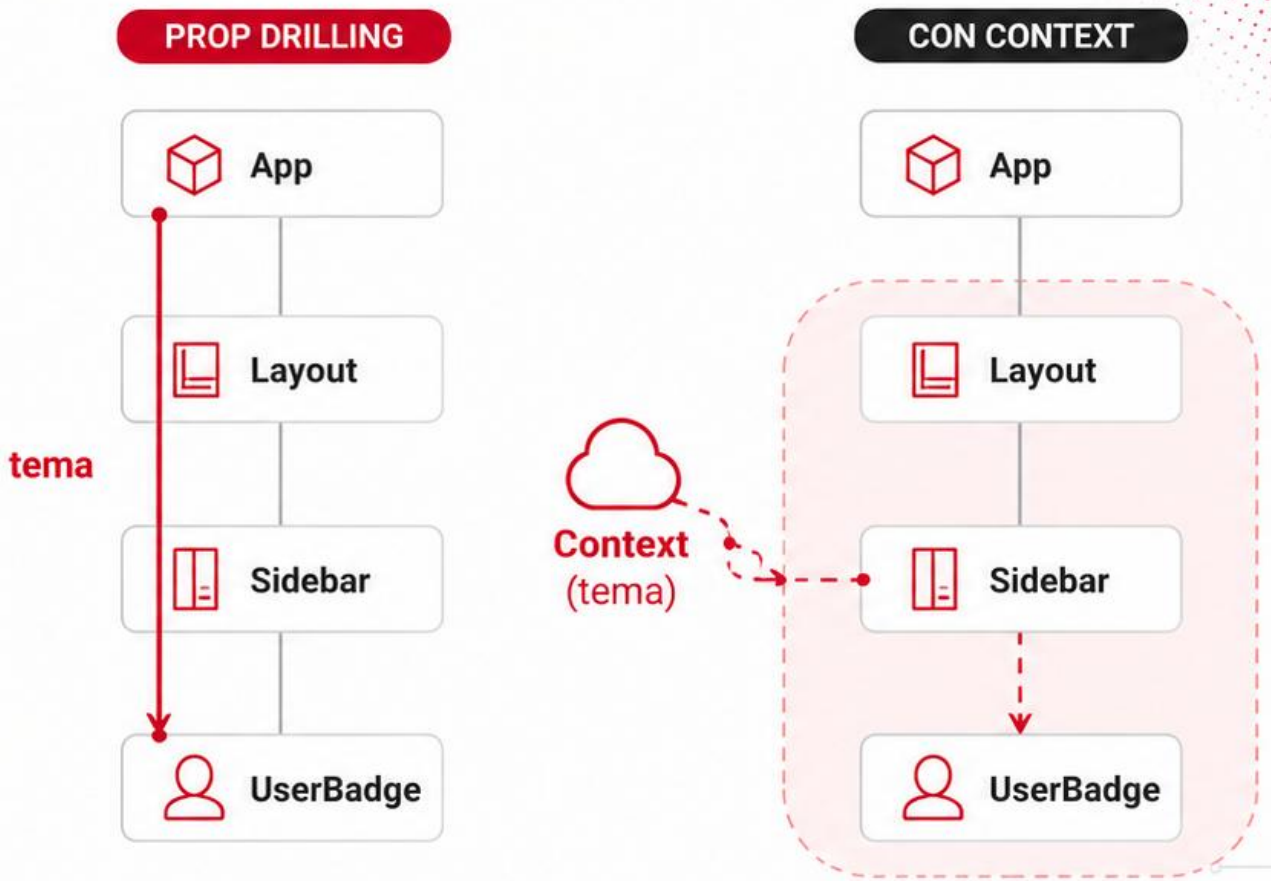
El problema del *prop drilling*

Pasar el mismo dato por capas que no lo usan

App → Layout → Sidebar → UserBadge.

Si “tema” o “usuario” debe atravesar componentes intermedios solo para llegar a un descendiente, aparece ***prop drilling***.

Context permite poner un valor a disposición de un subárbol sin reenviarlo manualmente por cada nivel.



createContext: define qué información puede compartirse

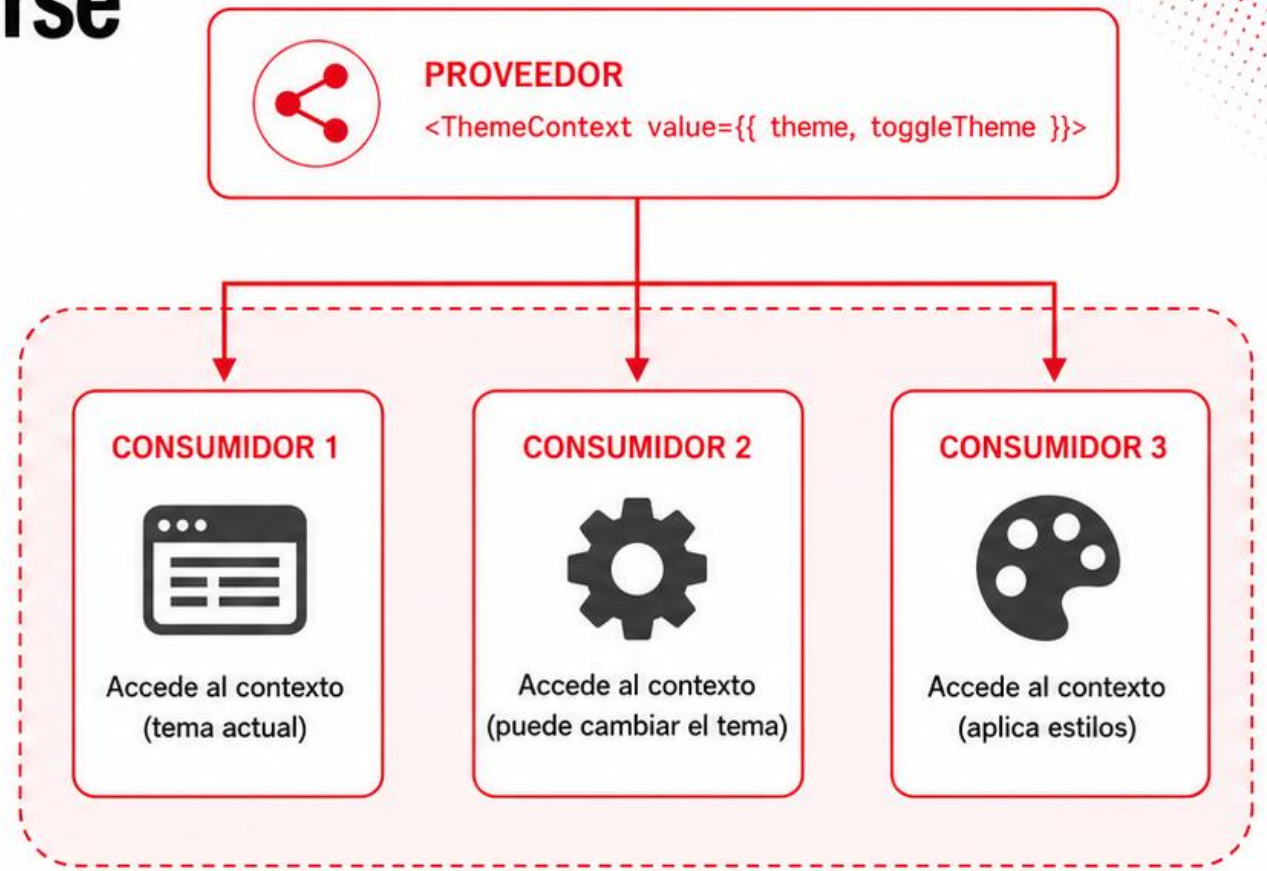
El contexto se crea fuera del componente

```
const ThemeContext = createContext(null);
```

- En React 19, el contexto puede usarse directamente como proveedor:

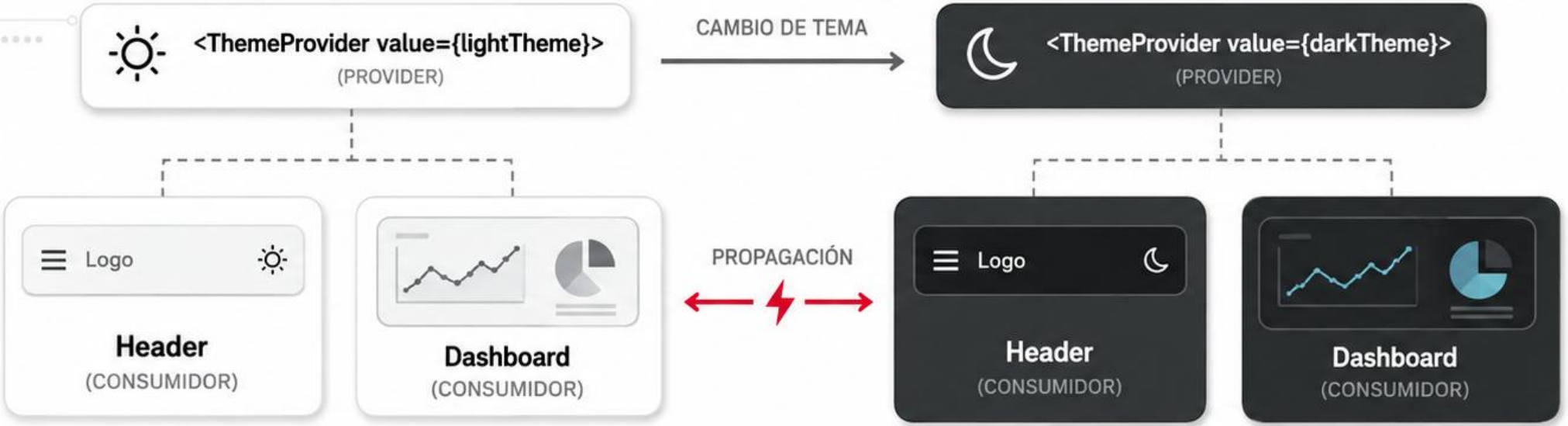
```
<ThemeContext value={{ theme, toggleTheme }}>
  ...
</ThemeContext>.
```

- El valor lo define un componente superior.



useContext: leer y suscribirse al valor compartido

El consumidor recibe el provider más cercano




```
const { theme, toggleTheme } = useContext(ThemeContext);
```

cuando cambia `value`, React vuelve a renderizar los consumidores de ese contexto.

Caso: Header y Dashboard reaccionan al mismo tema sin reenviar props por varias capas.

Props o Context: decide por alcance

La solución más global no siempre es la mejor

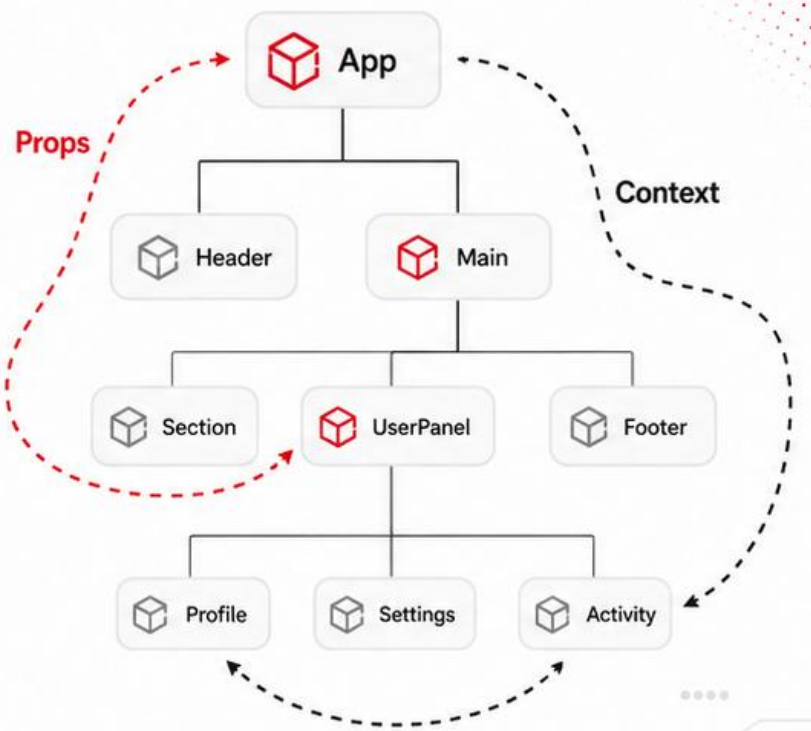
Usa props cuando: relación padre→hijo es directa, dato específico del componente, dependencia conviene que sea explícita.

Considera Context cuando: muchos niveles consumen el mismo valor, dato es transversal al subárbol, pasar props intermedias no aporta significado.



Ejemplos:

- Tema
- Usuario autenticado
- Idioma



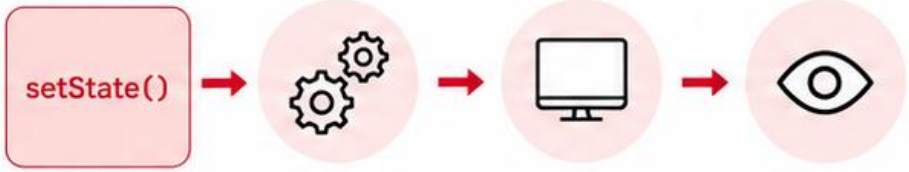
Universidad Tecnológica del Perú

useRef: persistir una referencia sin re-render

current puede cambiar sin actualizar la pantalla

```
const searchRef = useRef(null);
```

State → re-render



Ref → cambia current, sin re-render



- **useRef** devuelve el **mismo objeto** entre renders.
- Cambiar **searchRef.current** **NO provoca re-render.**
- **Usos comunes:** nodo DOM · id de temporizador · valor interno que no participa del JSX.

Demostración: enfocar el buscador con una ref

DOM real, controlado desde un evento

FLUJO DOM



1. `const searchRef = useRef(null);`
2. `<input ref={searchRef} ... />`
3. `searchRef.current?.focus();` al pulsar “Ir al buscador”.

Resultado: el cursor salta al input sin guardar el nodo en state.

UI REAL DE EJEMPLO



State vs Ref: la pregunta decisiva

¿El cambio debe verse en la UI?

- Si un cambio debe modificar el JSX visible → **state**.
- Si necesitas conservar una referencia o dato interno sin render → **ref**.



Universidad Tecnológica del Perú

“Resource Hooks” en el sílabo: equivalencia actual

React documenta Resource APIs y la función use(resource)

- Documentación actual de React: **Resource APIs** permiten acceder a recursos sin guardarlos como *state*.
- API principal: **use(resource)**.
- Puede leer un **Promise** o un **contexto**.
- **IMPORTANTE: use** no es técnicamente un Hook; por eso puede llamarse en condiciones o bucles, aunque debe usarse dentro de un componente o Hook.

MAPA DE TERMINOLOGÍA

Sílabo: resource Hooks → **React actual:** Resource APIs → **use(resource)**

TIPOS DE RECURSO

- **Promise** (asíncrono)
- **Context** (sincrónico)

NOTA TÉCNICA

use no es técnicamente un **Hook**; por eso puede llamarse en condiciones o bucles, aunque debe usarse dentro de un **componente** o **Hook**.

Universidad Tecnológica del Perú

use(resource): leer un Promise estable

El componente espera el recurso mediante Suspense




```
const avisos = use(announcementsPromise);
```

- el Promise debe ser **estable/cached** entre renders.
- En esta semana se usa un Promise local definido fuera del componente; **NO se usa fetch**.
- Si el Promise está pendiente, el **componente suspende** y React muestra el fallback de Suspense más cercano.

Recurso estable / cached
El Promise debe ser estable y se reutiliza entre renders.

NO se usa fetch
En esta semana se usa un Promise local definido fuera del componente.

Suspense: qué ve el usuario mientras espera

Una frontera de carga alrededor del componente que usa el recurso.

```
<Suspense fallback={<p>Cargando avisos...</p>}>
  <AnnouncementPanel resource={announcementsPromise} />
</Suspense>
```

ANTES: recurso pendiente



DESPUÉS: recurso resuelto



- El **fallback** aparece mientras el componente suspende.
- Cuando el recurso se resuelve, React reemplaza el **fallback** por el contenido.

Dos errores frecuentes con recursos

Promise nueva en render y try/catch alrededor de use

ERROR 1

```
function Panel() {
  const data = use(new Promise(() => {}));
  return <div>{data}</div>;
}
```

- **ERROR 1:** `const data = use(new Promise(...))` dentro del componente → recurso nuevo en cada render.

ERROR 2

```
function Panel({ promise }) {
  try {
    const data = use(promise);
    return <div>{data}</div>;
  } catch (e) {
    return <p>Error</p>;
  }
}
```

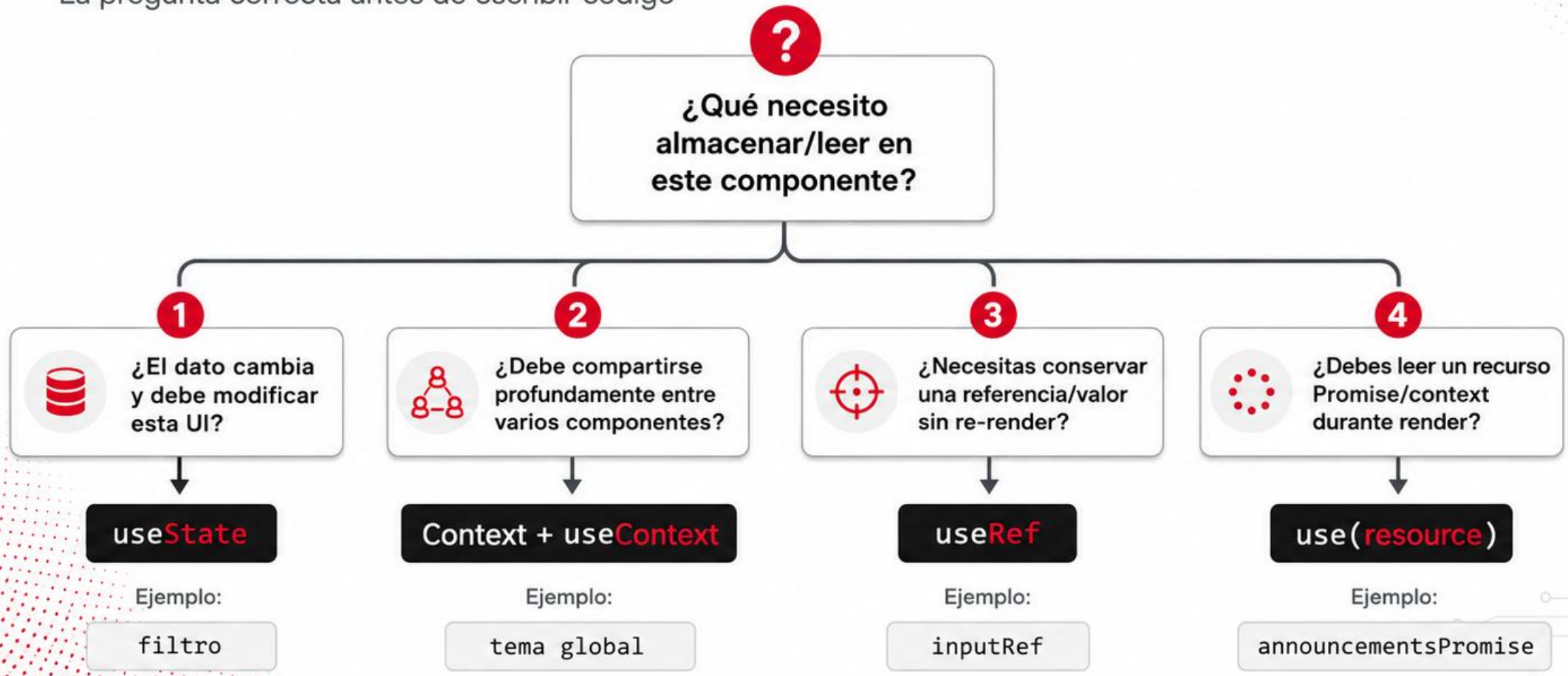
- **ERROR 2:** envolver `use(promise)` con `try/catch` → rompe la integración con Suspense.

```
const resource = announcementsPromise;
<Suspense fallback={<p>Cargando...</p>}>
  <AnnouncementPanel resource={resource} />
</Suspense>
```

- **Correcto:** recurso estable + Suspense; errores se manejan con Error Boundary cuando corresponda.

Mapa de decisión: state, context, ref o resource

La pregunta correcta antes de escribir código



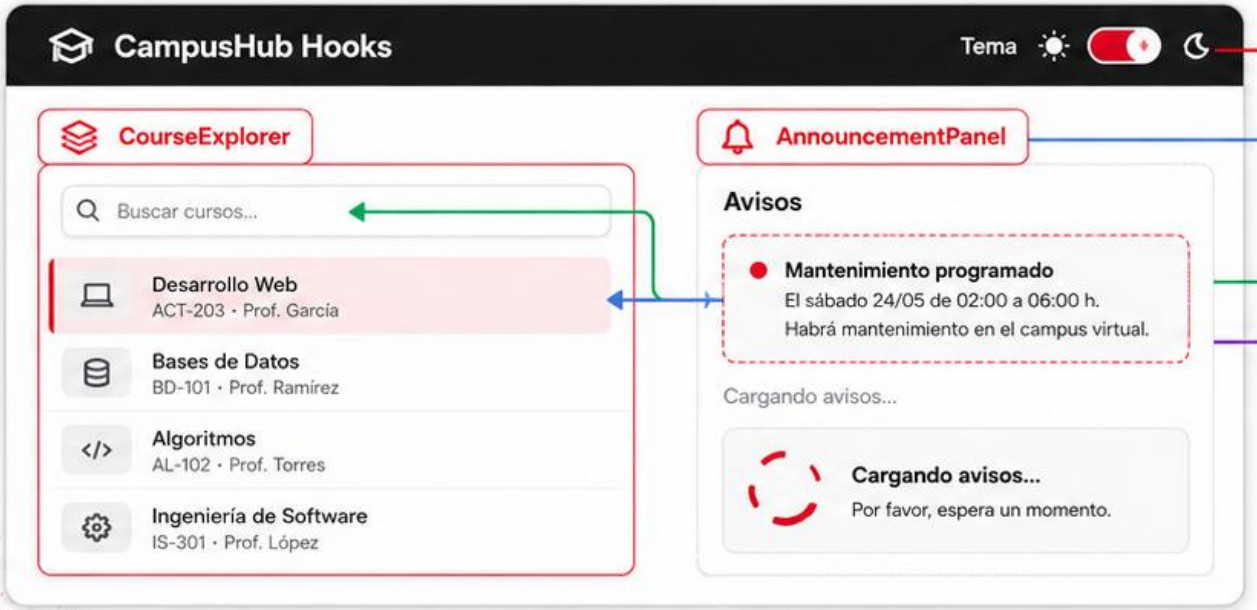
Universidad Tecnológica del Perú

Demostración integrada: CampusHub Hooks

Un mismo producto, cuatro responsabilidades distintas.

CampusHub Hooks: **useState** → filtro y búsqueda visible · **useContext** → tema global · **useRef** → foco del buscador · **use(resource)** → avisos locales bajo Suspense.

Evidencias: la lista cambia, el tema se propaga, el foco se mueve y el aviso aparece tras el fallback.



useContext
Tema global (ThemeContext)

useState
Filtro y búsqueda visible

useRef
Foco del buscador

use(resource)
Avisos locales bajo Suspense

- Tema se propaga a toda la app.
- La lista se filtra y cambia al buscar.
- El foco se mueve al campo de búsqueda.
- El aviso aparece tras el fallback.

Microcaso: elige la herramienta

Cuatro decisiones, treinta segundos cada una

A) Modal abierto/cerrado que cambia la UI. B) Idioma usado por muchas páginas. C) Botón “Enfocar correo”. D) Catálogo local entregado como Promise estable y mostrado bajo Suspense. Responde: state · context · ref · resource y justifica en una frase.



A) Modal abierto/cerrado que cambia la UI.

Respuesta: _____



B) Idioma usado por muchas páginas.

Respuesta: _____



C) Botón “Enfocar correo”.

Respuesta: _____



D) Catálogo local entregado como Promise estable y mostrado bajo Suspense.

Respuesta: _____

Universidad Tecnológica del Perú

Reto INDIVIDUAL: ServiceDesk Board

INDIVIDUAL

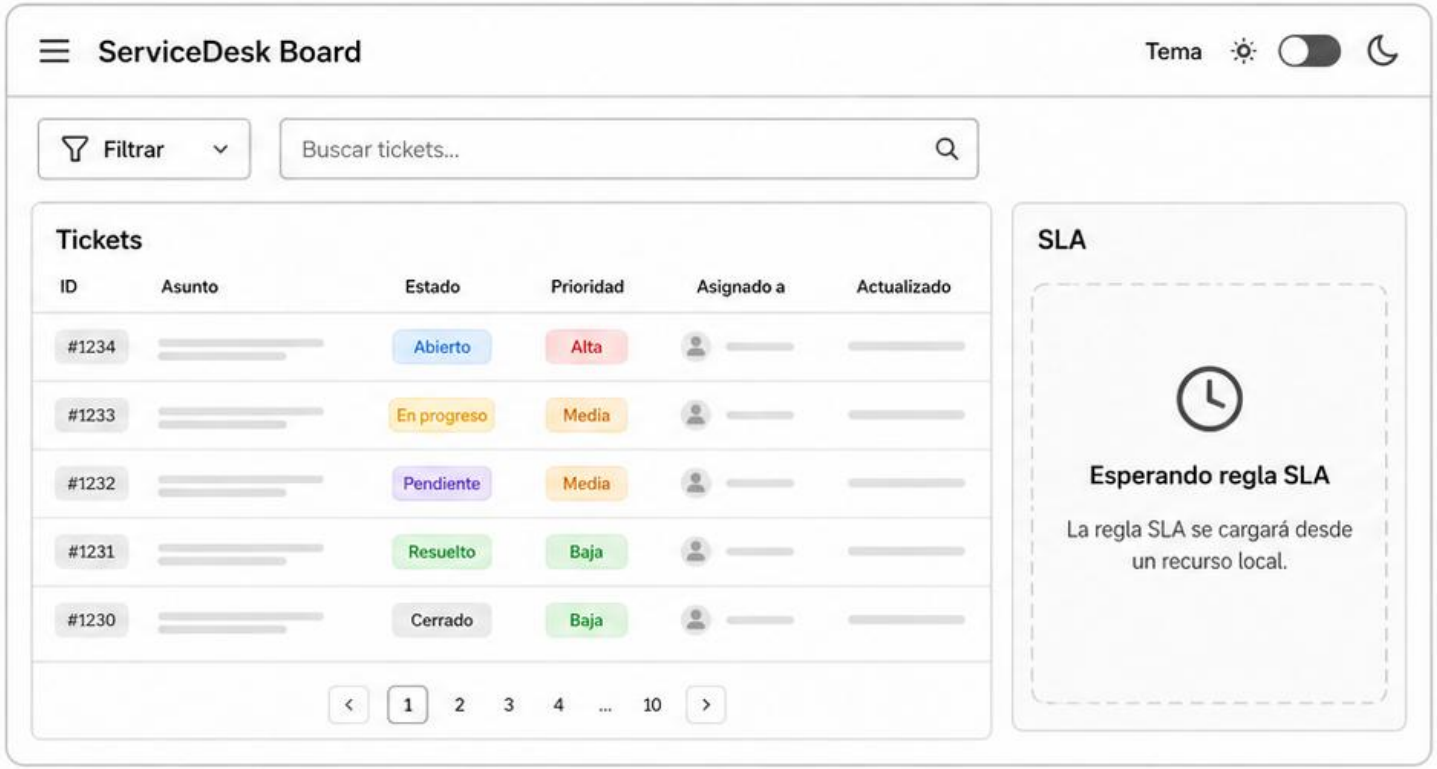
Construye una interfaz que use cada herramienta con una responsabilidad real

Contexto:
una mesa de ayuda necesita un tablero de tickets. Debe filtrar lo visible, compartir tema, enfocar el buscador y mostrar una regla SLA que llega como recurso local.

Modalidad:
INDIVIDUAL.

Tiempo total:
60 minutos.

Producto:
aplicación React ejecutable + evidencia de pruebas.



Producto mínimo y restricciones

Qué debe funcionar y qué no corresponde todavía

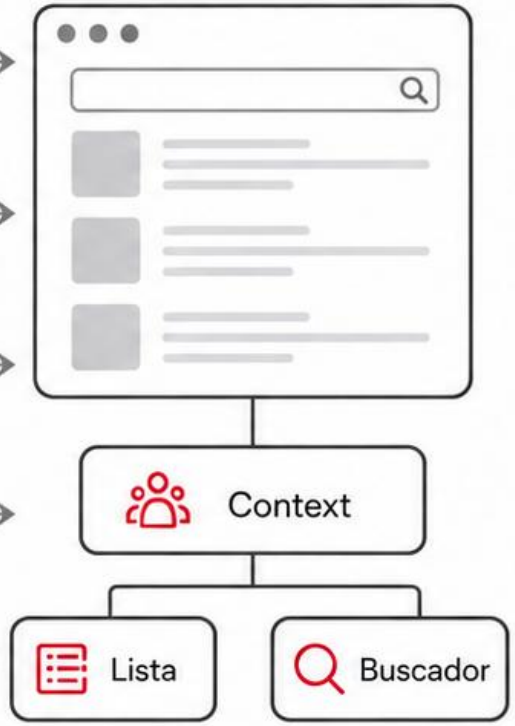
1) **useState** modifica lista visible;

2) **Context** + **useContext** afecta al menos dos componentes;

3) **useRef** permite enfocar el buscador desde un botón;

4) **use(resource)** lee **slaPromise** bajo Suspense;

5) sin errores en consola.



NO esta semana

- ✕ NO Fetch
- ✕ NO Axios
- ✕ NO Alova
- ✕ NO solución copiada del demo.

Construye, prueba y justifica

45 minutos de trabajo autónomo

INDIVIDUAL · Construcción autónoma. Ciclo recomendado: decide responsabilidad → implementa → ejecuta → observa → corrige → prueba otra interacción.

Si te bloqueas, formula una pregunta concreta: “¿qué debería provocar render aquí?” o “¿este valor debe compartirse o solo persistir?”.



45 min

Si te bloqueas, formula una pregunta concreta:

- ¿qué debería provocar render aquí?”
- ¿este valor debe compartirse o solo persistir?”

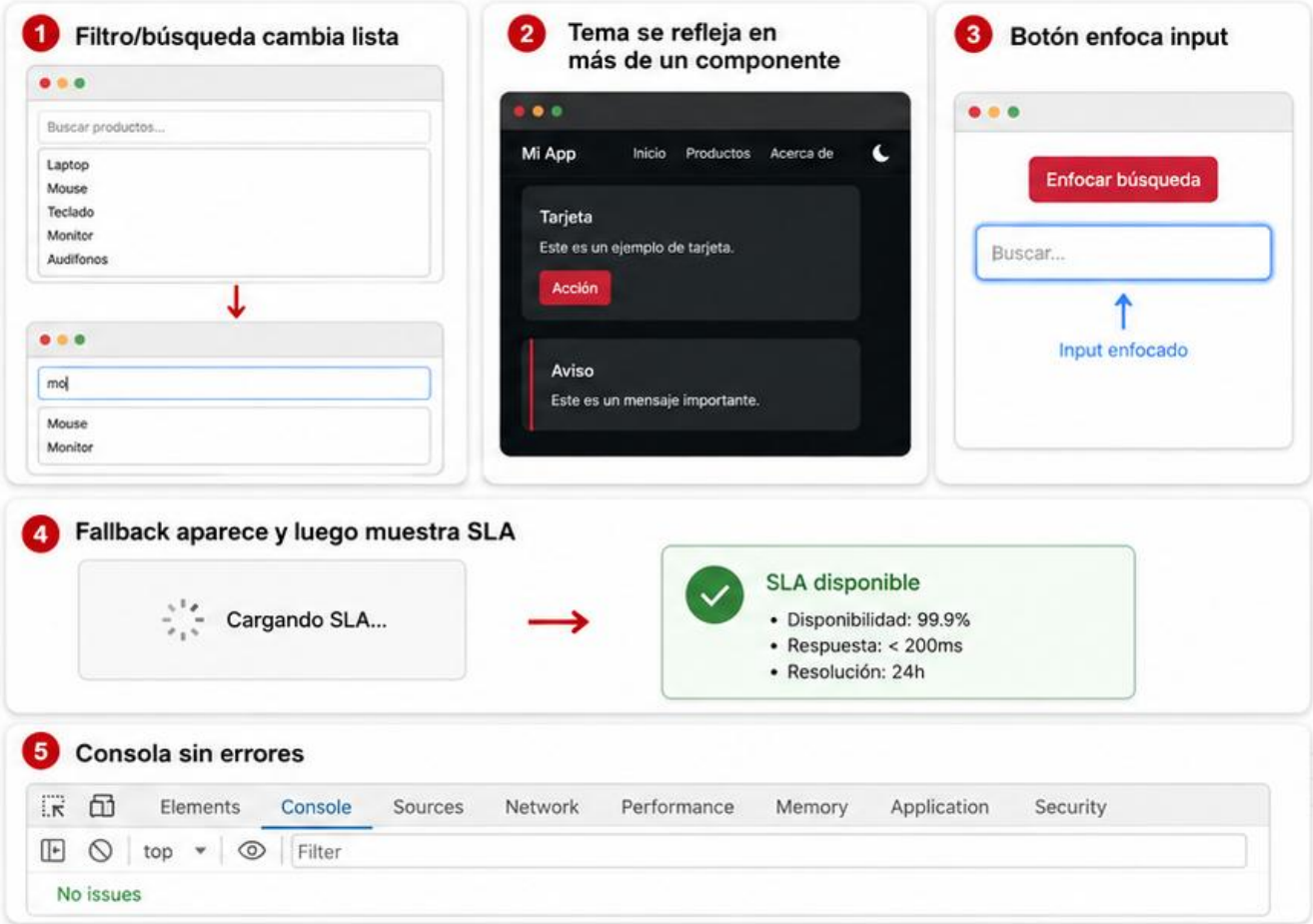
Universidad Tecnológica del Perú

Verifica antes de entregar

La aplicación debe demostrar los cuatro comportamientos

Checklist final:

- [ ] filtro/búsqueda cambia lista
- [ ] tema se refleja en más de un componente
- [ ] botón enfoca input
- [ ] fallback aparece y luego muestra SLA
- [ ] no hay Promise creada dentro del render
- [ ] no hay Hook dentro de if/for
- [ ] consola sin errores
- [ ] README/evidencia lista.



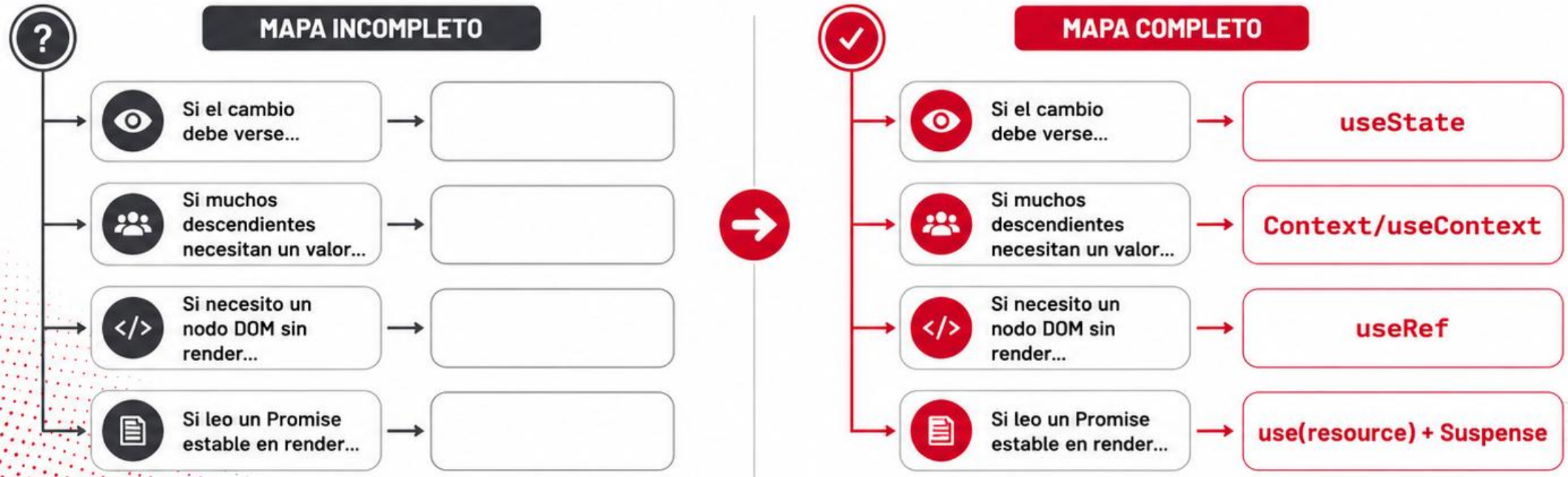
Universidad Tecnológica del Perú

Reconstruimos el mapa de decisiones

Primero las ideas del grupo, luego la síntesis

Completa oralmente: “Si el cambio debe verse…” → _____. “Si muchos descendientes necesitan un valor…” → _____.
“Si necesito un nodo DOM sin render…” → _____. “Si leo un Promise estable en render…” → _____.

`useState` · `Context/useContext` · `useRef` · `use(resource) + Suspense.`



Metacognición + ticket de salida

¿Puedes elegir antes de programar?

| Metacognición:

1. 1) ¿qué herramienta te resultó más fácil y por qué?
2. 2) ¿qué confusión tuviste entre state, context, ref o resource?
3. 3) ¿qué error evitarías en un próximo proyecto?

Ticket de salida

Ticket: *"Un dashboard debe recordar un filtro, compartir usuario, enfocar un campo y mostrar datos que llegan como Promise estable. Asigna una API a cada necesidad y justifica en una frase."*

Semana 14 → Servicios en React

Próxima semana: servicios en React con Fetch API, Axios y Alova.



¡MUCHAS GRACIAS POR SU ATENCIÓN!

Universidad Tecnológica del Perú